

Teamder — Store Release Readiness Audit

Round: audit only. Nothing was built, submitted, deployed, pushed, upgraded or reconfigured. **HEAD** `5b8f55c`, branch `perf/firestore-read-costs`.

Date of audit: 25.09.2026.

The headline

****iOS is not blocked, and the reason it looked blocked was mine, not the product's.****

The UIWindow crash is real, reproducible and fatal — but it is triggered by building against the **iOS 27 SDK**, which only Xcode 27 ships. EAS Build's `latest` image is **Xcode 26.6**. Apple's current requirement is ****Xcode 26 or later****, not 27. So the toolchain that actually builds our releases produces a binary that is both Apple-compliant and immune to the crash — which is exactly what iOS build 106 (1.1.14, 22.09) is, and it is on real phones running iOS 26.6.1 and 27.0 today without a launch crash.

I hit the crash because **my machine has Xcode 27**, and I built locally. That is a local-toolchain artefact, not a production blocker.

The real iOS finding is different and still serious: `image: "latest"` is a moving target. The day Expo promotes `latest` to Xcode 27, our next production build starts crashing at launch for every user, with no code change on our side and nothing in the repo to explain it.

And there is a bigger blocker nobody was looking at. The Firestore rules were tightened in production **this morning at 08:52 UTC** — before the client that goes with them shipped. The order is inverted, and it is already producing errors on the live binary.

1. Current release baseline

Branch / HEAD	perf/firestore-read-costs / 5b8f55c
Commits ahead of main	786
App version in app.json	1.1.14 — already live on both stores
Last Android build	1.1.14 vc242 , profile production , 22.09, FINISHED
Last iOS build	1.1.14 build 106 , profile production , 22.09, FINISHED
Working tree	not clean — see §17

Any new submission needs a version bump; 1.1.14 is taken.

Platform inventory (measured, not assumed)

Expo	^53 → 53.0.27 installed
React Native	0.79.6
React	19.0.0
EAS CLI	18.11.0 (on Node 20.20.2)
appVersionSource	remote — EAS owns buildNumber / versionCode
iOS bundle id	com.studiogameslime.soccerapp
Android package	com.studiogameslime.soccerapp
iOS deployment target	15.1 (Podfile + all 4 pbxproj configs)
Android minSdk / compileSdk / targetSdk	26 / 36 / 36
Wear targetSdk / compileSdk / vc	36 / 36 / 1027
CocoaPods	1.16.2
useFrameworks	static
Firebase JS SDK	firebase ^12.13.0
RN Firebase	@react-native-firebase/app + /messaging ^24.0.0
Functions runtime	Node 20
Native config plugins	19, incl. 8 local ones

2. Android readiness — READY

Google Play, [official target-API page](#):

"New apps and app updates must target Android 16 (API level 36) or higher to

be submitted to Google Play" — in force since **31 August 2026**.

We target **36**. Compliant, with no extension needed. Wear is at 36 too (its own requirement is only 35).

Nothing on the Android side blocks a submission. The blockers below are cross-platform.

3. iOS readiness — **READY**, with one configuration landmine

Apple, [Upcoming Requirements](#):

Since **28 April 2026** — "Apps uploaded to App Store Connect must be built

with **Xcode 26 or later** using an SDK for iOS 26, iPadOS 26, tvOS 26,

visionOS 26, or watchOS 26."

Since **9 September 2026** — "iOS and iPadOS apps uploaded to App Store

Connect must target iOS 13 or later."

We deploy at **15.1** — comfortably past the iOS 13 floor. And the Xcode 26 requirement is already being met, because `image: "latest"` resolves to Xcode 26.6 — which is *why* build 106 was accepted on 22.09.

No iOS 27 SDK requirement is announced. I checked Apple's requirements page specifically for one.

4. Exact UIWindowScene diagnosis

A. Why it happens. UIKit, when the binary is linked against the iOS 27 SDK, runs `_UIApplicationEvaluateRuntimeIssueForNoSceneLifecycleAdoption` during early initialisation — *before* `application(_:didFinishLaunchingWithOptions:)`.

If the app has no real `UIWindowSceneDelegate`, it aborts with SIGTRAP:

```
`Application failed to launch: UIWindowScene life cycle is required for apps built with this SDK.`
```

B. What kind of blocker it is — this is the question that mattered:

Simulator-specific?	No — it is the runtime check, device and simulator alike
Xcode-specific?	Yes — Xcode 27 only , because that is what ships the iOS 27 SDK
SDK-specific?	Yes — iOS 27 SDK only. A binary built against the iOS 26 SDK keeps the legacy lifecycle
App Store device blocker?	No, not today — Apple requires Xcode 26+, not 27
Guaranteed production crash?	Only if we ship an iOS-27-SDK build. We do not
Compatibility warning turned fatal?	Exactly that — it was a runtime issue, and the iOS 27 SDK makes it terminal

The evidence that settles it. iOS 1.1.14 is live and generating ordinary

Firestore errors from users on **osVersion 26.6.1** and **27.0**. An app that crashed before `didFinishLaunching` could not log anything. It launches.

The minimal `UIApplicationSceneManifest` already in `ios/Teamder/Info.plist` (`UIApplicationSupportsMultipleScenes: false`) is Expo's default. I tested adding to it earlier and it did **not** fix the Xcode 27 crash — a manifest without a real scene delegate is not adoption.

C. Minimal correct fix. A real `UISceneDelegate` plus a scene configuration in the manifest. Apple documents the migration in Technote **TN3187**.

D. Without an Expo/RN major upgrade? Not safely on Expo 53 / RN 0.79.

`expo-modules-core` and RN's `AppDelegate` own window creation on this version; scene adoption has to come from the framework, and it landed upstream in later RN. [expo/expo#46664](#) tracks it.

E. Smallest migration path, when it becomes necessary: Expo SDK ≥ 54 (EAS image `sdk-54` = Xcode 26.0) and the RN version that ships scene support, taken as one step with a full native QA pass — not a hand-rolled scene delegate on 53.

F. Blast radius, if we did it now: deep links (`application(_:open:)` moves to the scene), notification cold-open routing, Firebase/Google/Apple sign-in callback delivery, safe-area and keyboard geometry, Reanimated's native build, and every one of our 19 config plugins. Android is untouched. RTL is untouched.

G. Targeted adoption inside Expo 53 / RN 0.79: not recommended. It means patching framework-owned files and carrying the patch through every prebuild.

Verdict: DEFER — with a pin (\$5), not with hope.

5. EAS latest analysis — the real iOS finding

From [Expo's build infrastructure reference](#):

Alias	Image	Xcode
latest , sdk-57	macos-tahoe-26.5-xcode-26.6	26.6
sdk-56	macos-tahoe-26.4-xcode-26.4	26.4
sdk-55	macos-sequoia-15.6-xcode-26.2	26.2
sdk-54	macos-sequoia-15.6-xcode-26.0	26.0
sdk-53 (our SDK's default)	macos-sequoia-15.5-xcode-16.4	16.4

Two things follow, and they point the same way.

Our profile sets image: "latest" explicitly, and that is load-bearing.

Expo SDK 53's *default* image is Xcode **16.4** — which Apple has rejected for uploads since 28.04.2026. Without that explicit latest , our builds would be refused at upload. So the line is not an oversight; removing it breaks us.

But latest is the one value that can turn fatal on its own. Every image on that list today is Xcode 26.x — compliant, and no UIScene crash. The moment Expo promotes latest to an Xcode 27 image, our next production build links against the iOS 27 SDK and every launch aborts. Nothing in our repo changes. Nothing warns us. The first signal is crash reports.

Recommendation: pin macos-tahoe-26.5-xcode-26.6 . It is what latest already resolves to, so the pin changes nothing about today's build — it only removes Expo's ability to move it under us.

Is pinning a valid solution or a deferral? Both, precisely:

- **Valid today.** Apple requires Xcode 26+. Xcode 26.6 satisfies it on its own terms; this is not a loophole and not an old toolchain.
- **A deferral of UIScene.** When Apple mandates the iOS 27 SDK — not announced — the pin stops being viable and the Expo/RN migration becomes required. Apple's historical cadence puts that around spring 2027, which is the window to plan the migration in.

⚠ I could not read back which image build 106 actually used — eas build:view returned no image metadata. Xcode 26.x is an inference from Apple's requirement plus the upload having been accepted, not something I read off the build.

6. Firestore rules — the actual blocker, and it is already live

This is the item I most want you to look at.

****The production ruleset is `439be14e-f6e0-48da-ad3b-32480613ed8d` , released 25.09.2026 at 08:52:16 UTC**** — this morning, from the emergency security hotfix. It already contains the `/users` tightening:

```
match /users/{uid} {
  allow get:  if isSelf(uid) || isFullAccount();
  allow list: if isFullAccount();
}
```

The client that goes with it has not shipped. Store binary 1.1.14 still reads rosters through `hydrateUsers` / `hydratePlayers` on `/users` . HEAD moved `gameStore.hydratePlayers` to `groupService.hydratePublicUsers` — on 24.09, two days *after* 1.1.14 was built.

****The planned order was: binary → adoption → rules. What happened was rules first.**** It is already visible in production:

operation	version	platform	count	firstSeen	lastSeen (UTC)
<code>hydrateUsers</code>	1.1.14	ios	1	25.09	25.09 12:06
<code>hydratePlayers</code>	1.1.14	ios	1	25.09	25.09 12:06

Both `permission-denied` , both first seen *today*, both after the 08:52 deploy.

Volume is low, and the reason is luck, not design. `isFullAccount()` only denies *anonymous* sessions, and the silent-guest feature is not shipped — so only the 416 legacy anonymous accounts (34 active/7d) can hit it. When the new binary ships, **every fresh install gets an anonymous session**, and the old binary's blast radius stops being the relevant number.

This inverts the release argument. The onboarding release is not a feature drop that unblocks a future rules deploy — the rules are already deployed, and **the release is the repair.**

Repo rules go further than production

`firestore.rules` at HEAD is **not** what is live. Still undeployed:

match	live	repo
/users/{uid} create	isSelf(uid)	isFullAccount() && isSelf(uid)
/rounds/{id} read	isSignedIn()	isFullAccount()
/playerStats/{uid} read	isSignedIn()	isFullAccount()
/ratings/{uid} read	isSignedIn()	isFullAccount()
/users/{uid}/seasonTitles/... read	isSignedIn()	isFullAccount()
/pairStats/{pair} read	isSignedIn()	isFullAccount()

That is the remaining rules round. It must **not** go out before the binary.

Safe rollout sequence

The usual order does not apply, because half of it already happened:

1. Ship the binary (it repairs the live inversion).
2. Watch `hydrateUsers` / `hydratePlayers` `permission-denied` **fall to zero**
 - that is the adoption signal, and it is already instrumented.
3. Consider raising `minimumSupportedVersion` once adoption plateaus.
4. **Only then** deploy the repo's remaining `isFullAccount()` tightening.

No rules were deployed, changed or staged in this round.

7. Existing-user update compatibility — no regression

You asked whether an update can show an existing user the new onboarding by mistake. The gate is:

```
hasCompletedOnboarding: () => {
  const u = get().currentUser;
  return !!u && u.onboardingCompleted === true;
},
```

and `RootNavigator:435` sends anyone failing it to `PostSignInOnboardingScreen`.

Measured against production `/users`:

	count	what happens on update
Total user documents	709	
<code>onboardingCompleted == true</code>	660	nothing — straight in
<code>onboardingCompleted == false</code> (present)	39	shown onboarding — correct : these people genuinely never finished it
field absent	10	would be shown onboarding — but see below

The 10 with the field absent are **the known test-fixture stubs**, not people.

I read five of them back by id:

```
dt_admin      ['achievements','newGameSubscriptions','updatedAt']
rg_caller     ['achievements','newGameSubscriptions','updatedAt']
rgtest_owner  ['achievements','updatedAt']
```

No name, no email, no auth account behind them.

So: no real user who completed onboarding will be asked again. The read deserializer defaults `onboardingCompleted: d.onboardingCompleted === true`, and 660 of 660 real completed accounts carry the flag — including accounts created back in May. There is no pre-flag legacy cohort.

⚠ The 39 `false` accounts have names *and* emails, and 11 were created 19–23.09. They are a live onboarding-funnel drop, not an update-path problem — worth a separate look, out of scope here.

8. Auth / anonymous users

- Anonymous provider: **enabled** in production.
- `enableImprovedEmailPrivacy: true` — so `fetchSignInMethodsForEmail` returns `[]`, and guest→email upgrade must go through `accounts:signUp` (verified in the earlier Phase 0.3B probe), never `accounts:update`.
- The silent-guest flow, `pendingAction`, drafts and the contextual auth walls are all at HEAD and unreleased — ****no production runtime evidence exists for any of them.**** This is the largest untested surface in the release.

Android runtime QA of these journeys is possible today and is ****required before submission**** (§11). iOS runtime QA is possible too — the blocker was my local Xcode 27, and a cloud build or an Xcode-26 simulator sidesteps it.

9. Node 20 / Firebase Functions

`functions/package.json` pins `"node": "20"`. Node 20 is decommissioned

30.10.2026 — about five weeks out.

It does not block this app submission. It blocks *function deployments* after the deadline. Since the rules round (§6 step 4) needs no function deploy, it does not block that either. Treat it as a separate infrastructure task with a real date.

10. App Check — not a blocker

`const ENFORCE_APP_CHECK = false;` in `functions/src/index.ts:136`, applied to

every callable. Enforcement is off, so the new onboarding's callables are not gated by it. Not touched.

11. Authorized domains — DEFER

Production authorized domains:

```
localhost, soccer-app-52b6b.firebaseio.com, soccer-app-52b6b.web.app,  
teamderfc--landing-v2/v3/v4/preview-*.web.app, soccer-app-52b6b--landing-preview-*
```

`teamderfc.web.app` is **not** among them, and does not need to be: authorized

domains gate **web** OAuth redirects, and the app signs in natively. The invite

landing page contains **zero** Firebase references (`grep -c firebase`

`public/invite.html` → 0`) — it resolves an invite and deep-links.

Not a blocker. No config changed.

12. Analytics / crash reporting — a real visibility gap

signal	status
Crash reporting	NONE. No Crashlytics, no Sentry. <code>ErrorBoundary.tsx</code> says so in three comments
Error surface	in-app <code>logError</code> → Firestore <code>errors</code> — what this audit ran on
Auth failures	covered by <code>logError</code>
Onboarding funnel	<code>AuthCompleted</code> carries <code>action_kind</code> , <code>auth_method</code> , <code>is_existing_account</code> , <code>required_profile</code>
Guest→auth conversion	derivable from the above
Invite failures	covered

`logError` only fires for errors **JS catches**. A native crash — exactly the `UIScene` class — produces nothing at all. We would learn about it from store reviews.

Given this release is the first to carry anonymous guests, that gap is

RECOMMENDED to close, not a blocker. No analytics system was added.

13. Store metadata

item	status
Account deletion (Apple 5.1.1(v))	<code>DeleteAccountSheet.tsx</code> present
Sign in with Apple	<code>expo-apple-authentication</code> wired in <code>src/firebase/auth.ts</code>
Permission strings	present and in Hebrew (location, photos)
Screenshots	unverified — onboarding is a new first-run experience; current shots may not show it
Data Safety / App Privacy	unverified — anonymous accounts are a new data category

⚠ I did not open either store listing. Both unknowns are ****REQUIRED BEFORE SUBMISSION**** and need your eyes; §19 was an audit request and I am reporting what I could and could not determine from the repo.

14. Untriaged production errors

38 documents in `errors`, **all** `status: 'new'` — none triaged. The active one:

op	v	plat	count	note
govmapSearch	1.1.14	android	36	govmap autocomplete 403 — third-party 403, lastSeen today 16:17
uncaughtRender	1.1.14	android	4	"Rendered more hooks than during the previous render."
getTrustSummary	1.1.10	ios	18	permission-denied
getMyLiveOrUpcomingGames	1.1.10	android	95	permission-denied

govmapSearch is an external 403, not ours. **The hooks-order crash is ours**

and is a genuine render crash on the current live build — I did not chase it,
per your instruction not to fix.

15. Release decision table

Item	Android	iOS	Severity	Current status	Required action	Defer?	Risk if ignored
Rules deployed ahead of client	✗	✗	BLOCKER	live since 25.09 08:52	ship the binary	no	grows from 34 users to every fresh install
Version bump	✗	✗	BLOCKER	1.1.14 taken	→ 1.1.15	no	upload rejected
Working tree not clean	✗	✗	BLOCKER	QA tool uncommitted	commit or remove	no	unreproducible build
image: "latest" unpinned	n/a	⚠	REQUIRED	Xcode 26.6 today	pin macos- tahoe- 26.5- xcode- 26.6	no	silent launch crash on a future build
Guest/auth runtime QA	✗	✗	REQUIRED	zero runtime evidence	§16 checklist	no	the release's whole point is untested
Store metadata (shots, data safety)	✗	✗	REQUIRED	unverified	review both listings	no	review rejection
targetSdk 36	✓	n/a	ok	compliant	none	—	—
iOS deployment target 15.1	n/a	✓	ok	past the iOS 13 floor	none	—	—
Xcode 26 SDK requirement	n/a	✓	ok	met via latest	none	—	—
UIScene adoption	n/a	⚠	CAN DEFER	not required by Apple	plan SDK 54+ migration	yes	fatal when Apple mandates iOS 27 SDK
Remaining repo rules	✗	✗	CAN DEFER	undeployed	after adoption	yes	—
Node 20 EOL 30.10.2026	—	—	CAN DEFER	pinned to 20	runtime upgrade	yes (5 weeks)	no function deploys after the date
Crashlytics	✗	✗	RECOMMENDED	none	add a reporter	yes	native crashes invisible
teamderfc.web.app domain	—	—	CAN DEFER	not authorized	none	yes	—

Item	Android	iOS	Severity	Current status	Required action	Defer?	Risk if ignored
App Check	—	—	CAN DEFER	enforcement off	none	yes	—
38 untriaged errors	x	x	RECOMMENDED	none triaged	triage	yes	hooks-order crash stays live

16. QA matrix — release smoke checklist

Environment. A **release-signed** build is required for: Google/Apple sign-in (native callbacks need the release signature), deep links and App Links, and push. A dev build is sufficient only for: RTL, layout, keyboard and safe-area, and the mock-mode screens.

#	Journey	Android	iOS	needs release build
1	Fresh install → Guest Home	☐	☐	no
2	Guest → create club → auth → complete	☐	☐	yes
3	Guest → create game → auth → complete	☐	☐	yes
4	Guest → join game → auth → resume	☐	☐	yes
5	Guest → join club → auth → resume	☐	☐	yes
6	Guest → availability → auth → resume	☐	☐	yes
7	Guest → filler → auth → resume	☐	☐	yes
8	Existing user → no onboarding	☐	☐	yes
9	New user → minimal profile	☐	☐	yes
10	Game invite	☐	☐	yes
11	Club invite	☐	☐	yes
12	Personal invite /i/	☐	☐	yes
13	Invalid / dead invite	☐	☐	yes
14	Deep link cold start	☐	☐	yes
15	Deep link warm start	☐	☐	yes
16	Notification open	☐	☐	yes
17	Logout → login	☐	☐	yes
18	Update from the live store build	☐	☐	yes
19	Guest → upgrade/link account	☐	☐	yes
20	credential-already-in-use fallback	☐	☐	yes
21	PendingAction survives the auth detour	☐	☐	yes
22	Reinstall → anonymous account is NOT reused	☐	☐	yes

#18 and #22 matter most. #18 is the only test of the real upgrade path.

#22 matters because `footy.pending.action` is **device-scoped, not UID-scoped**

— it survives an account switch by design, and reinstall behaviour has never been exercised.

iOS QA is **no longer deferred** and needs no migration: build on EAS

(Xcode 26.6) and run on a device or an **Xcode 26** simulator. Do not QA on the local Xcode 27 toolchain — that is what produced the false blocker.

17. Exact blockers

Must be resolved before a submission can even be built:

1. **Ship the binary — the rules are already tight in production.** Not a feature release; a repair of an inverted rollout.
2. **Bump the version.** 1.1.14 is live on both stores.
3. **Clean the working tree.** `src/dev/QARouteLauncher.tsx` (new, 474 lines) and a 12-line addition to `src/navigation/RootNavigator.tsx` are uncommitted. Both gates (`__DEV__` and `EXPO_PUBLIC_QA_ROUTES==='1'`) mean it compiles out of a release build, and `.env.local` is restored — but a release must not be cut from a dirty tree. ****Your call: commit it as a dev-only utility, or remove it.****

Must be done before submitting:

1. Pin the EAS iOS image.
 2. Run the §16 matrix on both platforms with a release-signed build.
 3. Verify store metadata — screenshots and data-safety disclosures.
-

18. Recommended sequence

- **Phase A — decide.** QA tool: keep or remove. Pin the image. Bump to 1.1.15.
(Nothing here is started without your approval.)
- **Phase B — no platform migration.** Explicitly skipped: no Expo upgrade, no RN upgrade, no UIScene work. The pin covers it.
- **Phase C — Android QA.** Release-signed internal build, §16 matrix, #18 from the live store build.
- **Phase D — iOS QA.** EAS build (Xcode 26.6) → TestFlight → device.
Sign-in ×2, universal links, notifications, RTL, keyboard/safe-area, upgrade.
- **Phase E — release builds.** Build **last**, then diff the build's commit against HEAD before claiming anything shipped.
- **Phase F — submission.** Play production; App Store version + submit via the ASC API (`eas submit -p ios` only reaches TestFlight).
- **Phase G — staged rollout.** Android staged; hold iOS until Android is clean.
- **Phase H — monitoring.** Watch `hydrateUsers` / `hydratePlayers` `permission-denied` **go to zero**. That is the adoption metric.

- **Phase I — rules.** Only after H plateaus, deploy the repo's remaining `isFullAccount()` tightening.
- **Phase J — separate track.** Node 20 → 22 before 30.10.2026.

19. Blast radius of each required change

change	files	radius
Version bump	<code>app.json</code>	none — EAS owns the build numbers
Pin EAS image	<code>eas.json</code> (1 line)	iOS builds only; same image as today
Commit/remove QA tool	<code>src/dev/QARouteLauncher.tsx</code> , <code>src/navigation/RootNavigator.tsx</code>	none at runtime — double-gated, compiles out
Rules round (later)	<code>firestore.rules</code>	wide — 6 collections; needs the serial rules suite
Node 22 (later)	<code>functions/package.json</code> + CI	all 67 functions redeploy
UIScene (much later)	Expo + RN + all 19 plugins	largest — full native re-QA

20. Direct answers

****A.** Can the current Android codebase be prepared for store submission without a major platform migration?**

YES. `targetSdk 36` meets the requirement in force since 31.08.2026; `minSdk 26` and `compileSdk 36` are consistent; the last production build (vc242) succeeded on 22.09. Nothing Android-side needs a platform migration — only the version bump, a clean tree, and QA.

B. Can the current iOS codebase be submitted safely as-is?

YES, with one change I would not skip. The toolchain already satisfies Apple's Xcode 26 requirement, the deployment target clears the iOS 13 floor, and build 106 is live and launching on iOS 26.6.1 and 27.0. The change is pinning the EAS image so `latest` cannot move to Xcode 27 under us. Submitting without the pin is safe *today* and a coin-flip on the date of the next build.

****C.** Is pinning an older EAS iOS image a valid release solution, or only a temporary workaround?**

Valid now, and a deferral of UIScene — and the framing "older" is wrong.

Evidence: Apple requires **Xcode 26 or later** (28.04.2026). `latest` is

Xcode 26.6; every alias from `sdk-54` up is Xcode 26.x. Pinning

`macos-tahoe-26.5-xcode-26.6` pins to *what we already use* — not to an old toolchain, and not a loophole. What it does not survive is Apple mandating the iOS 27 SDK; that is unannounced as of today, and is the trigger for the migration, not for this release.

⚠ Note the one gap I could not close: `eas build:view` returned no image metadata, so "build 106 used Xcode 26.x" is inferred from Apple's upload requirement, not read off the build record.

D. Is an Expo/RN migration required before this store release?

NO. It is required before the first build made with the iOS 27 SDK.

Smallest safe path when that day comes: Expo SDK ≥ 54 with its matching RN, as one step, with a full native QA pass — not a hand-rolled scene delegate on 53.

E. Minimum set of changes required before we can submit:

1. Bump `app.json` version 1.1.14 → **1.1.15**.
2. Resolve the working tree — commit or remove the dev QA launcher.
3. Pin `eas.json` production `ios.image` to `macos-tahoe-26.5-xcode-26.6`.
4. Run the §16 matrix on both platforms with release-signed builds — especially #18 (update from the live build) and #22 (reinstall).
5. Verify store metadata: screenshots showing the new first-run, and data-safety disclosures covering anonymous accounts.

That is five items. **One is a code change, two are one-line config edits, and two are QA.** No platform migration, no dependency upgrade, no rules deploy.

Stopping here, as instructed. Nothing was built, submitted, deployed, pushed, upgraded, or reconfigured. `.env.local` remains as it was before the UI audit; the only working-tree changes are the two QA-tool files listed in §17.